

Organic Bitcoin (OBTC): A Living Monetary Network

Version: 0.1 (Draft)

Status: Request for Comment (RFC)

Audience: protocol engineers, wallet developers, mining pools, exchanges, researchers

Abstract

Organic Bitcoin (OBTC) is a Bitcoin-derived blockchain that treats coins as *living UTXOs* which must periodically move to remain valid. Every UTXO has a fixed **expiry horizon** of **T = 7 years** (measured in block height). When a UTXO reaches expiry, the protocol allows miners to **reap** it via a system transaction: **30%** of the UTXO value is routed to the block reward (security budget), and **70%** is **returned to the original locking script** (resetting the 7-year timer). OBTC further streamlines state growth by encouraging pruning: ordinary full nodes need retain only ~7 years of blocks; archival nodes preserve full history. The design aims to (i) recycle permanently lost coins into ongoing security, (ii) enforce periodic circulation (“organic metabolism”), and (iii) lower storage burdens while preserving trust via snapshot commitments.

1. Philosophy & Rationale

1.1 Organic money

Traditional Bitcoin prioritizes *immobility* as a feature: coins can remain inert forever. OBTC takes the opposite stance: **immobility is entropy**—lost keys, deceased owners, abandoned outputs. By **metabolizing** long-dormant UTXOs, OBTC turns dead supply into sustained security, while letting active users remain unaffected: simply move (or consolidate) before expiry.

1.2 Design goals

- **Sustained security budget:** recycle long-dormant supply into miner revenue, especially beyond halvings.
- **Activity hygiene:** enforce periodic UTXO consolidation and address rotation.
- **Operational lightness:** enable 7-year prunable nodes plus verifiable UTXO snapshots for fast sync.
- **Predictability:** simple, deterministic rules; minimal governance surface.
- **Neutrality:** scripts are treated uniformly; at expiry, *any* script may be reaped.

1.3 Non-goals

- Catering to indefinite cold storage and very long locks (>7y).
- Preserving historical payload data on all nodes. (Archival nodes handle that.)

2. Monetary Mechanics (High-Level)

Let $T = 7$ years and $\rho = 0.70$ be the return ratio at expiry (tax $\tau = 1 - \rho = 0.30$). For UTXOs that are never moved again ("permanently lost"), the repeated 7-year reaping implies a **geometric decay**. This is equivalent to a continuous annualized decay on that lost subset of supply of:

$$p = -\frac{\ln \rho}{T} \quad (\text{per year}).$$

With $T = 7$, $\rho = 0.7 \Rightarrow p \approx 5.1\%$ applied **only** to permanently lost coins. If the lost-supply share is L , the expected annual contribution to the security budget is roughly $B \approx L \cdot p$. For $L \in [20\%, 30\%]$, this yields **~1.0–1.5%/year** of total supply recycled into miner revenue.

3. Consensus Changes (Normative)

Scope: Hard fork relative to Bitcoin.

3.1 Expiry definition

- **Time base: block height** (deterministic). Define **blocks-per-year** $Y = 52,560$ (10-min target), thus **expiry distance** $E = T \cdot Y = 7 \cdot 52,560 = 367,920$ blocks from the **creation height** of each UTXO.
- A UTXO becomes **Expired** at height $h \geq h_{create} + E$.

3.2 Reaping permission

- Once Expired, a UTXO can be **spent without satisfying its script** by a special **system transaction** included by the block producer (miner). This transaction type is called **EUTXO-REAP**.

3.3 Return and taxation

- For each reaped UTXO of value v :
- **Returned value:** $v_{ret} = \lfloor \rho \cdot v \rfloor$ created to the **original locking script** (exact same `scriptPubKey`). This output's **expiry timer resets** (new creation height is the current block).
- **Taxed value:** $v_{tax} = v - v_{ret}$ contributes to the **block reward** for this block.
- **Dust return rule:** if $v_{ret} < \text{DUST}(h)$, the returned output **MUST NOT** be created; set $v_{ret} = 0$ and $v_{tax} = v$. (Avoids dust proliferation.)

3.4 Coinbase value limit (reward accounting)

- Let `BaseSubsidy(h)` be the chain's block subsidy schedule; `Fees(h)` be summed fees in block; and `ReapTax(h)` be the sum of v_{tax} across all EUTXO-REAP entries in the block.
- The coinbase transaction's total output value **MUST NOT exceed** `BaseSubsidy(h) + Fees(h) + ReapTax(h)`.

- **Maturity:** The additional `ReapTax` is subject to standard coinbase maturity (e.g., 100 blocks) before spendable.

3.5 Deterministic selection & throttling (anti-MEV)

- **Per-block cap:** At most **N** expired UTXOs can be reaped in a single block, and total REAP weight **MUST NOT** exceed **R vbytes**. (`N` and `R` are consensus parameters.)
- **Global order:** All expired UTXOs are ordered by the tuple (**creation_height ASC, txid ASC, vout ASC**). Miners **MUST** take the first eligible entries in that order until caps are hit. This removes discretion and MEV races.
- **No partial cherry-picking:** Skipping an eligible item to take a later one is invalid if capacity remains.

3.6 Script neutrality

- At expiry, the script is not evaluated. Multisig, P2SH, P2TR, timelocks, covenant-like constructions are treated identically.

3.7 Replay protection & chain identity

- Distinct address version bytes / Bech32 HRP and a unique `chain_id`-style flag for signatures to prevent cross-chain replays.

3.8 Activation

- **Fork point:** Bitcoin height **600,000** (deterministic snapshot). OBTC rules apply from OBTC-block 0 built on that UTXO set.

4. Data Structures & Node Behavior

4.1 Expiry index

Full nodes maintain an **Expiry Index**: a set of buckets keyed by **expiry height** $h_{exp} = h_{create} + E$. Each bucket stores outpoints `(txid, vout)` created at a common creation height window. On block connect:

1. For each new UTXO created, compute h_{exp} and append to the corresponding bucket.
2. For the current height h , pop the bucket h to retrieve newly Expired items (these become eligible in the global order defined in §3.5).

This avoids scanning history each block and makes REAP assembly $O(\text{capacity})$.

4.2 Pruning & archival

- **Prunable full node:** must retain the full UTXO set, the last **≈7 years** of blocks, and all headers. Blocks older than the rolling 7-year window may be pruned.
- **Archival node:** stores **all** historical blocks and transactions.
- **Invariant:** history is immutable; pruning never rewrites past headers or merkle commitments. Old payloads are simply not *stored* by prunable nodes.

4.3 UTXO snapshot commitments (fast sync)

Periodically (e.g., monthly), reputable nodes/wallets/pools may publish **UTXO set commitments** (e.g., Merkle/Verkle/Utreexo-style). A new node may: 1. Download a recent snapshot (state + proof), 2. Verify multiple independent commitments (multi-source), 3. Sync forward from that height.

This reduces initial sync cost while avoiding single-source trust.

5. Mempool & Mining Policy

5.1 Block template

A compliant miner assembles blocks as: 1. Collect ordinary transactions per feerate. 2. Determine eligible expired UTXOs via the global order (§3.5), add **EUTXO-REAP** entries up to caps N, R while enforcing dust rules. 3. Compute `ReapTax` and enforce coinbase value limit.

5.2 EUTXO-REAP encoding (wire format, sketch)

A canonical system transaction with: - `nVersion = 3` (reserved), `nType = REAP`. - **Inputs**: a vector of outpoints referencing expired UTXOs. These inputs **carry no scriptsigs/witnesses** and are valid **iff** each referenced UTXO is expired and uniquely selected per §3.5. - **Outputs**: for each input either (a) a single output back to the original `scriptPubKey` with value v_{ret} , or (b) omitted if below dust; plus **no explicit miner output** (tax is claimed via coinbase limit). - **Locktime/sequence**: fixed to zero. - **Weight accounting**: counted normally; caps N, R enforce DOS limits.

5.3 Rounding

`v_ret = floor(rho * v)` in satoshis. Any remainder increases `v_tax` to preserve conservation.

6. Wallet & UX Requirements

6.1 Core behaviors

- **Expiry awareness**: show a countdown for each UTXO; flag items with < 180 days.
- **One-tap renewal**: consolidate and re-issue to **new** addresses by default (privacy hygiene) resetting the 7-year timer.
- **Auto-renew service**: optional opt-in; wallet pre-signs policies to renew within a randomized window (e.g., 6–12 months before expiry) at a user-set max-feerate.
- **Dust warning**: if a UTXO's 70% return would fall below DUST, clearly warn the user.

6.2 Address/script policy

Consensus returns to the **original script**. Wallets **should** immediately forward that 70% to a **fresh script** as part of the user's renewal flow, to limit on-chain linkability.

6.3 Fee ergonomics

Renewal should target historically low-fee windows; wallets may schedule oracles that trigger when feerate < user threshold.

7. Dynamic Dust Threshold

7.1 Definition

Let $\tilde{f}(h)$ be the **30-day median feerate** (sat/vB) observed up to height h . Let v_{in} be a representative input virtual size (e.g., P2TR keypath spend ~68 vB). Define:

$$\text{DUST}(h) = \alpha \tilde{f}(h) \cdot v_{in}, \quad \alpha \in [1.0, 3.0].$$

Interpretation: the minimum output that would cost roughly $\alpha \times$ its own size to spend at typical fees.

Recommendation: $\alpha = 2$.

7.2 Consensus vs policy

`DUST(h)` is a **consensus parameter** updated *mechanically* from on-chain feerate medians (no governance vote). Update cadence: once per **2,016 blocks** (difficulty epoch) to limit volatility.

8. Economics & Game Theory

8.1 Security budget

Annualized security inflow from lost coins: $B \approx L \cdot (-\ln \rho / T)$. With $\rho = 0.7$, $T = 7$, and $L = 25\% \Rightarrow B \approx 1.27\%/y$.

8.2 Compliance incentive

Rational users renew if their **renewal cost fraction** φ (fees/value) is less than tax $\tau = 30\%$. Since $\varphi \ll 30\%$ for most UTXOs, compliance is high. For very small outputs, the dust rule prevents pathological churn.

8.3 MEV mitigation

Deterministic order + per-block caps `remove` discretion, preventing miners from cherry-picking only high-value expiries. This also smooths `ReapTax` variance and orphan risk.

9. Privacy Considerations

- Returning to the **same script** could link cycles. Wallets should auto-forward to fresh scripts.

- Renewal batching must avoid revealing user clusters; randomized schedules (6–12 months before expiry) help.
- Archive-light nodes should avoid leaking sync preferences; prefer multiple commitment sources.

10. Security Considerations & Attack Mitigations

- **DOS via expired set explosion:** mitigated by caps N , R , weight accounting, and bucketed expiry index.
- **Miner favoritism:** forbidden by deterministic order; block invalid if order is violated while capacity remains.
- **Time manipulation:** height-based expiry avoids MTP skew.
- **Replay across chains:** distinct HRP/prefix and signature domain separation.
- **Snapshot poisoning:** require multi-party commitments; clients verify from at least k independent sources before trusting.
- **Reorg safety:** ReapTax coinbase maturity (100 blocks) prevents instant spend; REAP entries are ordinary inputs for reorg semantics.

11. Governance & Upgrades

- **Hard-coded constants:** $T = 7$ years; $\rho = 0.70$; dust formula with $\alpha = 2$; coinbase maturity 100 blocks; blocks/year $Y = 52,560$.
- **Tunable via delayed activation** (minimal surface): per-block REAP caps N and size R ; dust multiplier α . Any change requires:
 - 2/3 client signaling over a full difficulty epoch,
 - 60-day grace period,
 - fixed activation height.
- No on-chain treasury or discretionary committees.

12. Implementation Plan (MVP)

1. **Consensus:** implement EUTXO-REAP, expiry index, deterministic ordering, dust rule, coinbase limit.
 2. **Node:** prunable mode (7-year window) + archival mode; UTXO snapshot export/import + commitment verification.
 3. **Wallet:** expiry UI, one-tap renewal, auto-renew scheduler, dust warnings, default post-return forwarding.
 4. **Mining:** block template logic; pool validation of REAP order; instrumentation for ReapTax analytics.
 5. **Tooling:** public dashboard—expiring supply ($\leq 90d$), renewal rate, ReapTax share, archive node map.
 6. **Testnets:** time-accelerated expiry (e.g., $T = 7$ days) for end-to-end rehearsal; adversarial tests for REAP ordering.
-

13. KPIs (Operational)

- `ReapTax / MinerRevenue` (annualized)
 - Expiring-soon supply (≤ 90 days)
 - Renewal success rate
 - Avg & p95 REAP count per block; variance
 - Archive node count & geo dispersion
 - Wallet auto-renew penetration; complaint rate
-

14. Parameterization Appendix (Math)

- **Annualized decay on never-moved coins:** $p = -\ln \rho / T$.
 - **Security inflow:** $B = L \cdot p$.
 - **Iso-budget transform:** Holding p constant, moving from (T, ρ) to (T', ρ') requires $\rho' = \exp(-p T')$.
 - **Design by target budget:** Given $B_{target}, L, T : \rho = \exp\left(-\frac{B_{target} T}{L}\right)$, tax $\tau = 1 - \rho$.
-

15. Glossary

- **Expiry:** the moment a UTXO reaches age T (in blocks) and becomes reappable without script satisfaction.
 - **EUTXO-REAP:** system transaction that reaps expired UTXOs, returning ρ to original script and taxing the remainder to miners.
 - `ReapTax`: cumulative taxed value in a block added to the coinbase limit.
 - **Prunable node:** retains recent ~7 years of blocks + full UTXO set.
 - **Archival node:** retains full history.
-

16. Legal & Compliance Note

OBTC is an independent chain and is **not** Bitcoin. The expiry recycling may be interpreted variably under different jurisdictions (as a protocol fee, negative carry, or demurrage on lost coins). Exchanges and custodians should review operational policies accordingly.

17. License

This document is released under the **CC BY 4.0** license. Reference implementations may be licensed under MIT.
